As well as improve your OpenAPI description, you can customize the code generation directly, as you'll see in the next section.

11.4.4 Customizing the generated code

In this section you'll learn about some of the customization options available with the NSwag generator and why you might want to use them. I look at three customization options in this section:

- Using System.Text.Json instead of Newtonsoft.Json for JSON serialization
- Generating an interface for the generated client implementation
- Not requiring an explicit `BaseAddress` parameter in the constructor

By default, NSwag uses Newtonsoft.Json to serialize requests and deserialize responses. Newtonsoft.Json is a popular, battle-hardened JSON library, but .NET 7 has a built-in JSON library, System.Text.Json, that ASP.NET Core uses by default for JSON serialization. Instead of using two JSON libraries, you may want to replace the serialization used in your client to use System.Text.Json.

When NSwag generates a client, it marks the class as `partial`, which means that you can define your own `partial class FruitClient` (for example) and add any methods that you think are useful to the client. The generated client also provides partial methods that act as hooks just before a request is sent or received.

DEFINITION Partial methods in C# (http://mng.bz/zXEB) are `void`-returning methods that don't have an implementation. You can define the implementation of the method in a separate partial class file. If you don't define the implementation, the method is removed at compile time, so you use partial methods as highly performant event handlers.